

Classifying Astronomical Objects into Stars/Galaxies/Quasars using Photometric Data

Arun Pravin A P · B Harshith Varma · Monish U

Amrita School of Computing
Amrita Vishwa Vidyapeetham, Coimbatore

Agenda

1. Introduction & Motivation
2. The SDSS DR17 Dataset
3. Exploratory Data Analysis
4. Preprocessing & Feature Engineering
5. Classification Algorithms
6. Results & Conclusion

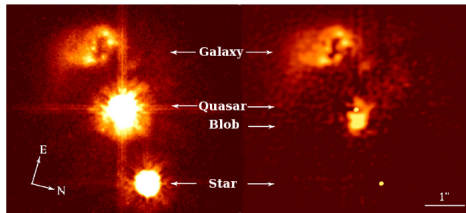
Goal: build a machine learning model that classifies an astronomical object as a **Star**, **Galaxy**, or **Quasar (QSO)** from its photometric measurements.

SDSS DR17 Astronomical Object Classification

The **SDSS Data Release 17 (DR17)** provides extensive astronomical observations. The dataset contains **100,000 objects**, each described by photometric, positional, and spectroscopic measurements.

Different celestial objects emit light with different properties - this is the signal our model will learn to recognise.

Objective: classify an astronomical object as a **Star**, **Galaxy**, or **Quasar (QSO)**, based on the properties of the light it emits.



*Example SDSS imaging: a galaxy, a quasar, and
a star*

SDSS DR17 Dataset – Attributes (1/2)

Attribute	Type	Description
obj_ID	float64	Unique identifier assigned to each astronomical object.
alpha	float64	Right Ascension (RA): the object's east–west position on the celestial sphere, in degrees.
delta	float64	Declination (Dec): the object's north–south position on the celestial sphere, in degrees.
u	float64	Apparent magnitude in the u-band filter (ultraviolet / near-UV wavelengths).
g	float64	Apparent magnitude in the g-band filter (green portion of the optical spectrum).
r	float64	Apparent magnitude in the r-band filter (red portion of the optical spectrum).
i	float64	Apparent magnitude in the i-band filter (red / near-infrared range).
z	float64	Apparent magnitude in the z-band filter (longest-wavelength SDSS band).

SDSS DR17 Dataset – Attributes (2/2)

Attribute	Type	Description
run_ID	int64	Identifier for the SDSS imaging run during which the object was recorded.
rerun_ID	int64	Identifier for the version of the SDSS image-processing pipeline used.
cam_col	int64	Identifier for the camera column that recorded the object's image.
field_ID	int64	Identifier of the specific imaging field the object was observed in.
spec_obj_ID	float64	Unique identifier for the object's spectroscopic observation.
class	str	Target variable: STAR, GALAXY, or QSO (quasar).
redshift	float64	Displacement of spectral features toward longer wavelengths; linked to motion and, for distant objects, cosmological distance.
plate, MJD, fiber_ID	int64	Identifiers for the spectroscopic plate, observation date, and optical fiber used to collect the object's light.

Photometric Attributes

Feature	Filter	Approx. wavelength	What it tells us
u	Ultraviolet	~355 nm	Brightness in near-ultraviolet light
g	Green	~475 nm	Brightness in the blue/green region
r	Red	~622 nm	Brightness in the red region
i	Infrared	~763 nm	Brightness in the deep-red / near-infrared region
z	Infrared	~905 nm	Brightness in the near-infrared region

Redshift measures how much an object's light has been stretched toward longer wavelengths. For distant astronomical objects, this stretching is primarily caused by the expansion of the universe – so redshift tells us about an object's motion and, for cosmological sources, its distance and look-back time.

Part A – Exploratory Data Analysis

Checking data quality and understanding feature behaviour

Missing Values

Since this is an official dataset extracted from NASA's SDSS repository, the dataset contains **no missing values**.

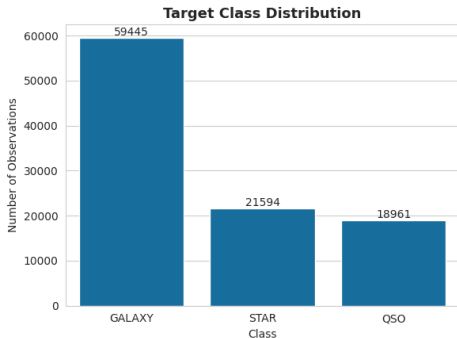
```
obj_ID .....0
alpha .....0
delta .....0
u, g, r, i, z .....0
run_ID, rerun_ID .....0
cam_col, field_ID .....0
spec_obj_ID .....0
class, redshift .....0
plate, MJD, fiber_ID .....0
```

Total missing values

0

out of 100,000 rows $\times$ 18 columns

Target Class Distribution



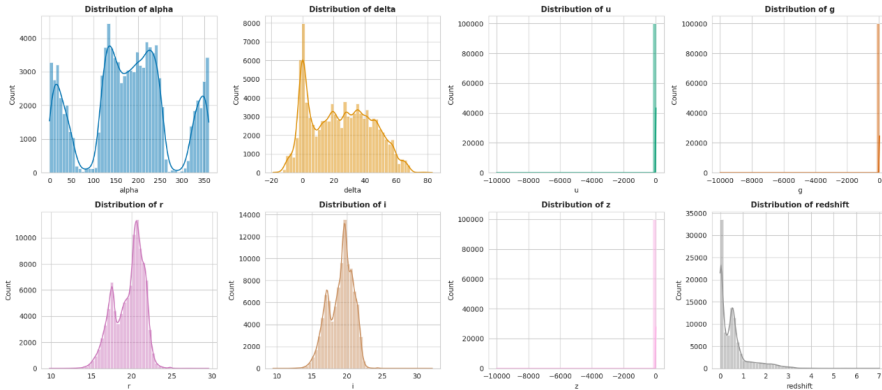
The dataset is **not perfectly balanced**:

- **GALAXY**: 59.44% (majority class)
- **STAR**: 21.59%
- **QSO**: 18.96%

This imbalance could cause a model to favour the majority class.

Fix: a **stratified 80/20 train-test split** preserves these same class proportions in both sets, so every model is trained and evaluated on a representative distribution of all three classes.

EDA Analysis – Feature Distributions



Interpreting the Feature Distributions

1. Sky coordinates

alpha, delta give the object's position in the sky.

- Broad coverage across the survey region
- No obvious extreme anomalies

2. Photometric magnitudes

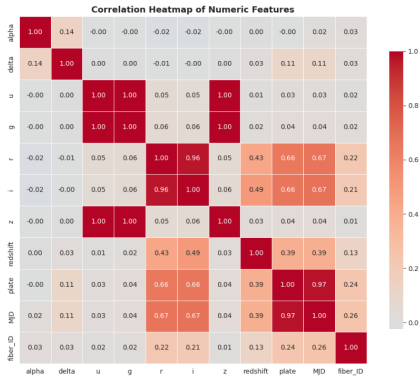
u, g, r, i, z are brightness through five optical filters.

- Concentrated around typical values
- u, g, z contain a sentinel −9999 value (only 1 row)
⇒ safely removed

3. Redshift

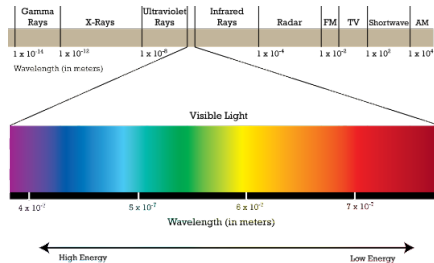
- Strongly **right-skewed**
- Most objects: small redshift
- Small tail of distant, high-redshift objects

Correlation Heatmap

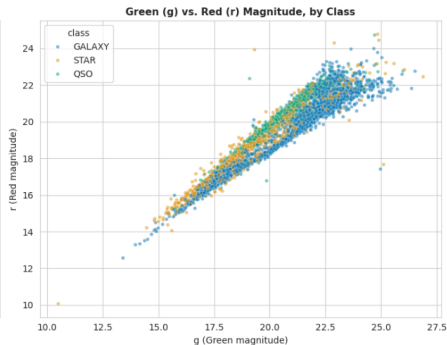
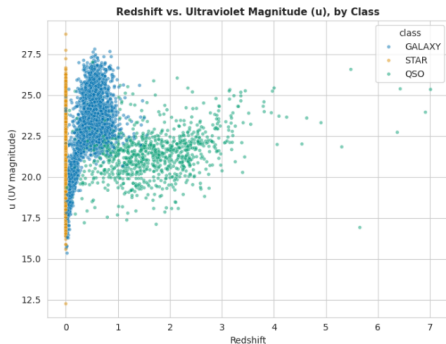


Interpreting the Correlation Heatmap

- **u**, **g**, **z** are almost perfectly correlated (~ 1.00) – they measure the *same* object through different filters, so overall brightness moves them together.
- **r** and **i** are strongly correlated (0.96) – nearby wavelengths track each other closely.
- **redshift** has moderate correlation with **r** (0.43) and **i** (0.49) – redshift shifts the light distribution across bands.



Feature vs. Target Relationships



What the Feature–Target Plots Tell Us

- **Redshift** shows clear class-dependent patterns – it particularly separates the low-redshift stellar population from the higher-redshift QSO population.
- **Photometric magnitudes** show strong relationships, but with considerable overlap between classes.

These plots motivated engineering **colour indices** – $u-g$, $g-r$, $r-i$, $i-z$ – which capture an object's *relative* spectral characteristics rather than only its absolute brightness.

Part B – Preprocessing and Feature Engineering

Cleaning, encoding, splitting, and engineering features

Data Cleaning

Data cleaning strategy (justified):

1. No duplicate rows or duplicate obj_ID were found $\Rightarrow$ no de-duplication necessary.
2. A small number of rows carry the SDSS **−9999 sentinel value** in the u/g/z bands, indicating a *failed* photometric measurement rather than a genuine reading. Since this affects a negligible fraction of 100,000 rows, these rows are **dropped** rather than imputed, avoiding a fabricated magnitude that could mislead the model.
3. rerun_ID is **constant** for every row and carries no discriminative information – it is removed along with the other identifier columns.

Exact duplicate rows: 0 | Duplicate obj_ID values: **21,947** | Rows with **−9999 sentinel**: 1

Encoding & Scaling

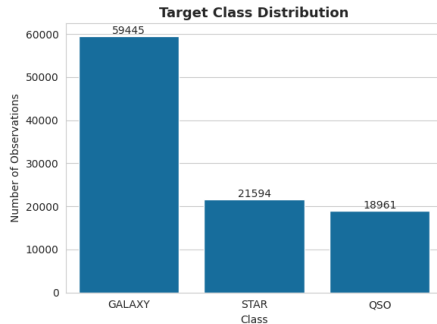
1. **Remove non-predictive identifiers.** Dropped 9 identifier/administrative columns (obj_ID, run_ID, rerun_ID, cam_col, field_ID, spec_obj_ID, plate, MJD, fiber_ID) – these describe *where/when/how* an observation was collected, not the object's physical properties.
2. **Encode the target.** class was label-encoded: GALAXY $\rightarrow$ 0, QSO $\rightarrow$ 1, STAR $\rightarrow$ 2.
3. **Standardization.** StandardScaler fitted *only* on the training set, then applied to both train and test sets – this prevents data leakage from the test set.

Columns retained for modelling: alpha, delta, u, g, r, i, z, class, redshift

Stratified Train–Test Split

- Dataset split into **80% training** (79,999 rows) and **20% testing** (20,000 rows).
- Stratification** preserves the same class proportions in both sets.

Class	Train	Test
GALAXY	59.4%	59.4%
STAR	21.6%	21.6%
QSO	19.0%	19.0%



Feature Engineering – Colour Indices

We engineer **colour indices** because the raw u , g , r , i , z values only tell us how bright an object appears in each band. Instead, we take the *difference* between neighbouring bands (e.g. $u-g$, $g-r$) to capture whether an object looks relatively *bluer* or *redder*. Since stars, galaxies, and quasars have different light patterns, these differences give the model more discriminative information.

New engineered columns: u_g , g_r , r_i , i_z

	mean	std	min	max
u_g	0.0	1.0	-12.063	8.049
g_r	0.0	1.0	-18.064	18.104
r_i	0.0	1.0	-30.275	23.179
i_z	0.0	1.0	-31.910	31.047

Standard Scaling

We use `StandardScaler` because features live on very different numerical scales: `alpha/delta` are sky coordinates, `redshift` has its own range, and the engineered colour indices can span much larger ranges. Left unscaled, features with larger raw magnitudes would dominate the model simply due to *scale*, not importance. Scaling puts every feature on a common footing (mean ≈ 0 , std ≈ 1), so models compare and use features fairly.

Feature	mean	std	min	max
alpha, delta	0.0	1.0	-2.19	2.99
u, g, r, i, z	0.0	1.0	-5.47	7.43
redshift	0.0	1.0	-0.80	8.82
u_g, g_r, r_i, i_z	0.0	1.0	-31.91	31.05

Final Preprocessed Dataset

Columns fed into the classifiers: alpha, delta, u, g, r, i, z, redshift, u_g, g_r, r_i, i_z
(12 numeric features → 3-class target)

	alpha	delta	u	g	r	i	z	redshift	u_g	g_r	r_i	i_z
1	0.62	0.08	0.86	0.29	-0.12	-0.19	-0.23	-0.26	1.13	1.11	0.23	0.16
2	-0.02	-0.85	-1.53	-1.97	-2.18	-2.22	-2.25	-0.67	0.50	0.03	-0.25	0.17
3	1.84	-1.27	-0.84	-0.75	-0.46	-0.22	-0.02	-0.79	-0.30	-0.93	-0.92	-0.81
4	-0.03	-0.91	1.37	0.17	-0.30	-0.35	-0.34	-0.29	2.33	1.23	0.10	-0.04
5	-0.46	0.12	0.78	0.65	0.30	0.20	0.12	-0.20	0.36	1.03	0.43	0.30

This fully cleaned, scaled, and feature-engineered table is what gets fed into every classifier in the next section.

Classification Algorithms

Five models, one target: STAR / GALAXY / QSO

Logistic
Regression

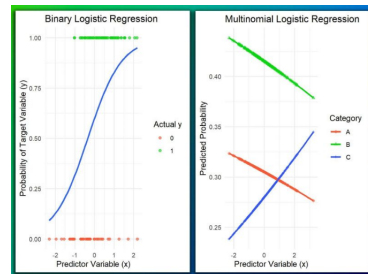
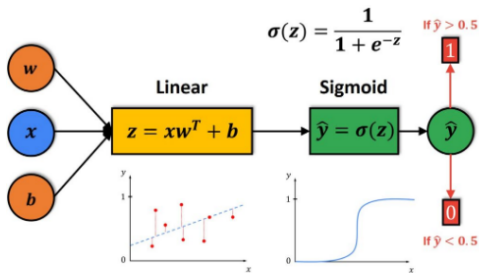
K Nearest
Neighbours

Gaussian
Naive Bayes

Decision
Tree

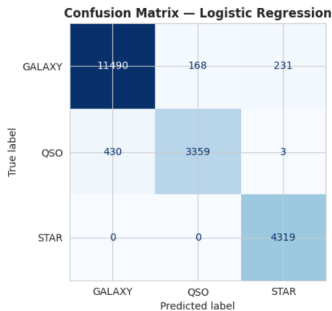
Support Vector
Machine

Logistic Regression



Logistic Regression models the probability of each class using a linear combination of features passed through a **sigmoid** (binary case) or **softmax** (multinomial case) function, giving well-calibrated class probabilities that sum to 1.

Logistic Regression – Test Set Results



Class	Prec.	Recall	F1
GALAXY	0.96	0.97	0.97
QSO	0.95	0.89	0.92
STAR	0.95	1.00	0.97

96% overall accuracy. All 4,319 STARs correctly classified; 11,490/11,889 GALAXYs correct. Main error source: **430 QSOs misclassified as GALAXY**, likely because low-redshift QSOs can resemble galaxies photometrically.

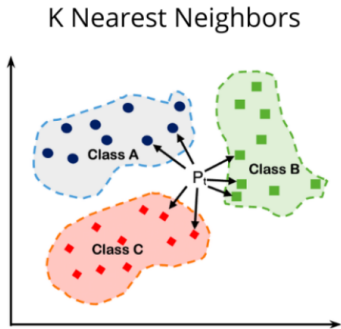
Interpreting Coefficients in Logistic Regression



- **Positive coefficient** → increasing that feature pushes the prediction *towards* that class.
- **Negative coefficient** → increasing that feature pushes the prediction *away* from that class.

Redshift dominates the coefficients, confirming the EDA observation that it is the strongest discriminator – particularly for separating STARS from QSOs.

K Nearest Neighbours (KNN)



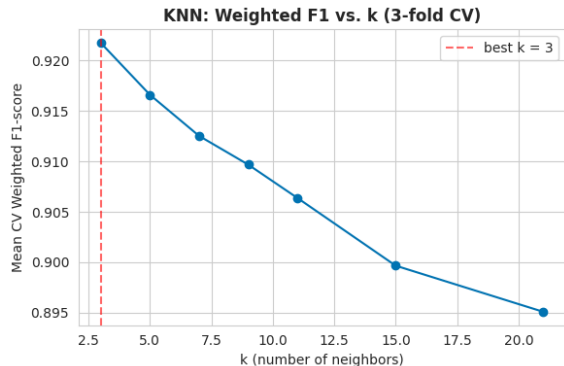
KNN classifies an object based on its nearest neighbours in feature space. For a new astronomical object, the algorithm:

1. Calculates its distance from the training objects.
2. Finds the K closest objects.
3. Assigns the class that occurs most frequently among those neighbours.

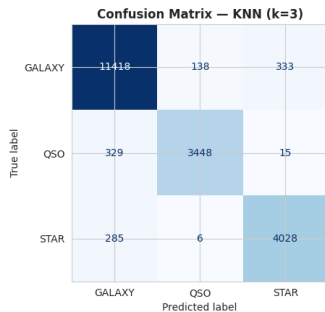
Choosing K

Tested $K = 3, 5, 7, 9, 11, 15, 21$ using **3-fold cross-validation** on a 15,000-sample subset (to keep tuning practical), selecting by **weighted F1-score**.

$K = 3$ achieved the highest mean CV weighted F1 (0.9218) $\Rightarrow$ selected for the final model.



KNN Results ($k = 3$)



Class	Prec.	Recall	F1
GALAXY	0.95	0.96	0.95
QSO	0.96	0.91	0.93
STAR	0.92	0.93	0.93
Overall accuracy: 94%			

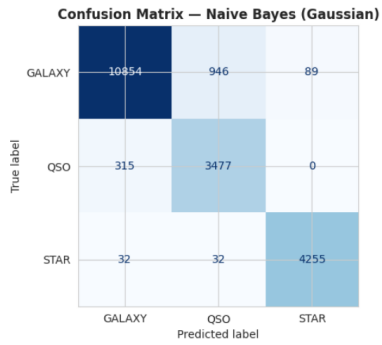
Gaussian Naive Bayes

Suppose we have an object with features $X = (x_1, x_2, \dots)$, where each x_i could be redshift, u , g , r , i , z , $u-g$, etc. The goal is to calculate, for each candidate class $C \in \{\text{STAR}, \text{GALAXY}, \text{QSO}\}$:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}$$

Naive Bayes assumes the features are **conditionally independent** given the class.

Naive Bayes – A Violated Assumption



The strong correlations seen in the EDA heatmap between u , g , r , i , z (and, by construction, the colour indices derived from them) **directly violate** Naive Bayes's conditional-independence assumption.

This is the most likely reason Naive Bayes **underperforms** relative to models that do not assume feature independence (Logistic Regression, Decision Tree, SVM) on this dataset.

What is a Decision Tree?

A **Decision Tree** is one of the most intuitive ML algorithms - it works exactly like a **flowchart of yes/no questions**.

Real-world analogy: imagine an astronomer trying to classify a new object. They might reason like this:

- **Q1:** Is the redshift very close to zero?
 - **Yes** → Likely a **STAR** (it's in our galaxy, not moving away)
 - **No** → Ask Q2...
- **Q2:** Is the redshift very high (e.g., > 0.5)?
 - **Yes** → Likely a **QSO** (quasars are extremely distant)
 - **No** → Likely a **GALAXY**

Decision Trees automate this reasoning: they learn which questions to ask, in what order, and where to place the cutoffs – all directly from the training data.

How Does a Decision Tree Learn?

At each step, the tree needs to decide: "Which feature should I split on, and at what value?"

The intuition: pick the question that best *separates the classes*. A good split leaves each resulting group as *pure* as possible (mostly one class).

Measuring purity - Gini Impurity:

$$\text{Gini} = 1 - \sum_{i=1}^C p_i^2$$

where p_i is the proportion of class i in a group.

- Gini = **0** → perfectly pure (all one class) → *best*
- Gini = **high** → classes all mixed up → *worst*

The algorithm's strategy: at each node, try every possible split on every feature, and pick the one that reduces impurity the most. Repeat until the branches are pure (or until a stopping condition is hit).

Why We Need to Control Tree Depth

A tree that keeps splitting forever will eventually create a *unique branch for every training object* - achieving 100% training accuracy. But this is a trap!

Analogy - The Over-Prepared Student:

- Imagine a student who memorises every practice problem *word-for-word*, including typos.
- They ace the practice test (100%)...
- ...but fail the real exam because the questions are slightly different.

This is Overfitting:

- The model learns the **noise** in training data, not the underlying **pattern**.
- Great training accuracy, poor test accuracy.
- **Solution:** limit tree depth (`max_depth`) so it learns *general rules*, not memorised rows.

Our approach: we tune `max_depth` by tracking both training *and* test accuracy, and pick the depth where test accuracy peaks – that's where the model generalises best.

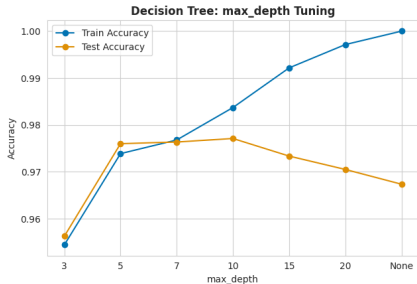
Decision Tree – Tuning `max_depth`

As `max_depth` increases, training accuracy steadily improves (from $\sim 95.5\%$ at depth 3 to nearly 100% unrestricted). **Test** accuracy improves only up to **depth 10** ($\sim 97.7\%$), then gradually decreases – the model begins **overfitting** beyond that point.

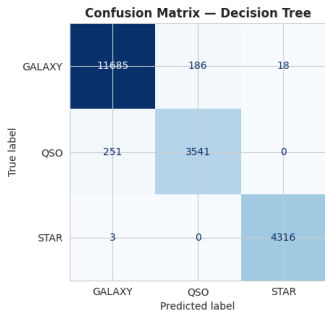
What the graph shows:

- The *gap* between train and test accuracy is small at depth 10 $\rightarrow$ healthy fit.
- As depth grows, the gap *widens* $\rightarrow$ classic overfitting signature.

`max_depth = 10` balances learning the underlying pattern against overfitting – selected for the final tree.



Decision Tree – Results



Class	Prec.	Recall	F1
GALAXY	0.98	0.98	0.98
QSO	0.95	0.93	0.94
STAR	1.00	1.00	1.00

Overall accuracy: 98%

Best performer so far. STARs are classified **perfectly**, and GALAXY/QSO both exceed 93% across metrics.



How to Read the Tree

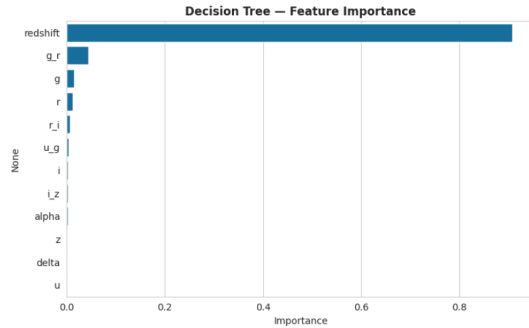
Each **box (node)** in the diagram tells you:

- **The question** being asked (e.g., $\text{redshift} \leq 0.004$)
- **Gini impurity** $\rightarrow$ how mixed the classes are at this node
- **Samples** $\rightarrow$ how many training objects reached this node
- **Value** $\rightarrow$ how many belong to each class (GALAXY / QSO / STAR)
- **Colour** $\rightarrow$ which class dominates (deeper colour = purer node)

How to trace a prediction: start at the root (top). At each node, if the condition is **true**, go **left**; if **false**, go **right**. Keep going until you land on a leaf (bottom) – the majority class in that leaf is the prediction.

Key insight: the very first split is almost always on **redshift**, confirming it as the strongest single discriminator for our three classes.

Decision Tree – Feature Importance



Interpreting Feature Importance

Feature importance measures how much each feature contributes to reducing impurity across *all splits in the tree*. Higher score = more useful for classification.

Key observations:

- **Redshift dominates by a huge margin** - this single feature does most of the work for separating STARS, GALAXYs, and QSOs.
- **Photometric bands (u, g, r, i, z) and colour indices (u_g, g_r, r_i, i_z)** play supporting roles, helping refine borderline cases.
- **Positional features (alpha, delta)** have near-zero importance - *as expected*, since an object's position in the sky shouldn't determine its physical type.

This validates our EDA: redshift's strong class-dependent behaviour observed in scatter plots is confirmed here as the model's most-used feature.

What is a Support Vector Machine (SVM)?

Imagine you have two groups of objects scattered on a piece of paper - red dots and blue dots. Your task: **draw a straight line that separates them.**

The question is: which line do you pick? There could be infinitely many valid lines.

SVM's answer: pick the line that stays *as far away as possible* from both groups.

- Not the line that just *barely* separates them
- But the one with the **largest safety buffer** on both sides

Why this matters: a line with a big buffer is more likely to correctly classify *new* points it hasn't seen before – giving SVM excellent generalisation power.

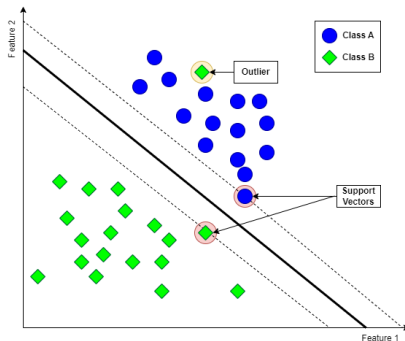
Understanding the Margin & Support Vectors

The **Margin** is the "safety buffer" - the distance from the decision line to the nearest point of any class.

- SVM finds the line that **maximises** this margin.
- A wider margin = more confidence in future predictions.

Support Vectors are the specific data points that lie *right on the edge of the margin* - the ones "supporting" the boundary.

- Only these points determine the boundary.
- Points far from the boundary can be removed with *no effect* on the model.



What if the Data isn't Linearly Separable?

Sometimes, no straight line can separate the classes - imagine one class forming a *circle* inside another.

The Kernel Trick: SVM can *transform* the data into a higher-dimensional space where a straight line (or plane) *can* separate them.

Analogy: imagine red dots forming a ring around blue dots on a flat table.

- In 2D → impossible to separate with a line.
- **Lift the blue dots up** into 3D (like pulling them up off the table) → now a flat plane can slice between them!

Common kernels:

- **Linear** - use when data can already be separated with a straight line
- **RBF (Radial Basis Function)** - use for complex, curved boundaries

Key SVM Hyperparameters - C and Kernel

Before training, we must choose two important settings:

1. C - The Regularisation Parameter

- Controls the trade-off between having a *wide margin* vs. *correctly classifying every training point*.
- **Small C** (e.g., 0.1) → wider margin, tolerates some misclassifications → more general
- **Large C** (e.g., 10) → narrower margin, tries harder to classify every point correctly → risk of overfitting

2. Kernel - The Boundary Shape

- **linear** - draws a straight boundary (fast, works if classes are already well-separated)
- **rbf** - draws curved boundaries (more flexible, but slower)

How to pick the best combination? Use `GridSearchCV` to systematically test different combinations and evaluate each with cross-validation.

SVM – Hyperparameter Tuning

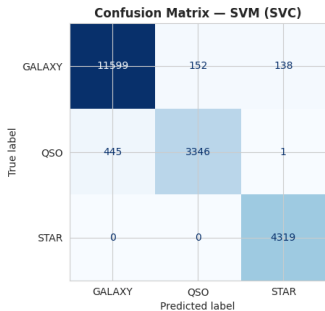
GridSearchCV was used on an 8,000-sample subset with 3-fold cross-validation to tune C and the kernel.

C	Kernel	Mean CV weighted-F1
10	linear	0.9601
10	rbf	0.9583
1	linear	0.9494
1	rbf	0.9491
0.1	linear	0.9309
0.1	rbf	0.8929

Best configuration: linear SVM with $C = 10$ (weighted F1 = 0.9601) – used to train the final model on the full training set.

Why linear won: after scaling and colour-index engineering, the three classes are already well-separated in the feature space – a straight boundary is enough. RBF's extra flexibility isn't needed and would

SVM – Results



Class	Prec.	Recall	F1
GALAXY	0.96	0.98	0.97
QSO	0.96	0.88	0.92
STAR	0.97	1.00	0.98

Overall accuracy: 96%

Strong all-round performance. STARs achieve 100% recall (all correctly caught), but QSO recall is the weakest at 88% - some QSOs still get confused with GALAXYs.

Overall Model Performance

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Decision Tree	0.9771	0.9771	0.9770	0.9771	0.9861
SVM (SVC)	0.9632	0.9632	0.9631	0.9632	0.9874
Logistic Regression	0.9584	0.9584	0.9584	0.9584	0.9865
KNN (k=3)	0.9447	0.9447	0.9449	0.9447	0.9703
Naive Bayes	0.9293	0.9293	0.9355	0.9293	0.9799

Note: Precision, Recall, F1-Score, and ROC-AUC are calculated using the **weighted average** across all three classes (STAR, GALAXY, QSO).

Sanity Check – Are We Over-Relying on Redshift?

Our initial results were unusually strong, with the Decision Tree reaching **97.71% accuracy**. Since EDA and feature importance both showed that **redshift** was extremely predictive, we asked an important question:

Are the models genuinely learning from the photometric features, or are they simply relying on redshift?

To test this, we performed a **redshift removal experiment**: we created a separate modelling pipeline, completely removed **redshift**, and retrained the classifiers using the remaining features.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
SVM (SVC)	0.8754	0.8736	0.8754	0.8734	0.9519
KNN ($k = 5$)	0.8704	0.8681	0.8704	0.8685	0.9382
Decision Tree	0.8672	0.8652	0.8672	0.8649	0.9390
Logistic Regression	0.7586	0.7474	0.7586	0.7378	0.8416

What Does Removing Redshift Tell Us?

With redshift

- Decision Tree accuracy: **97.71%**
- SVM accuracy: **96.32%**
- Redshift was the dominant feature
- Decision Tree performed best overall

Without redshift

- Best accuracy: **87.54%** (SVM)
- KNN: **87.04%**
- Decision Tree: **86.72%**
- SVM becomes the best-performing model

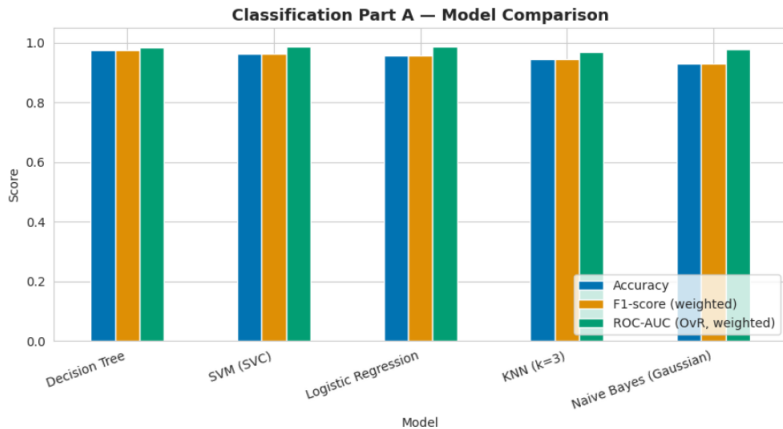
Interpretation

Removing redshift causes a clear performance drop, confirming that **redshift contains very strong class information**.

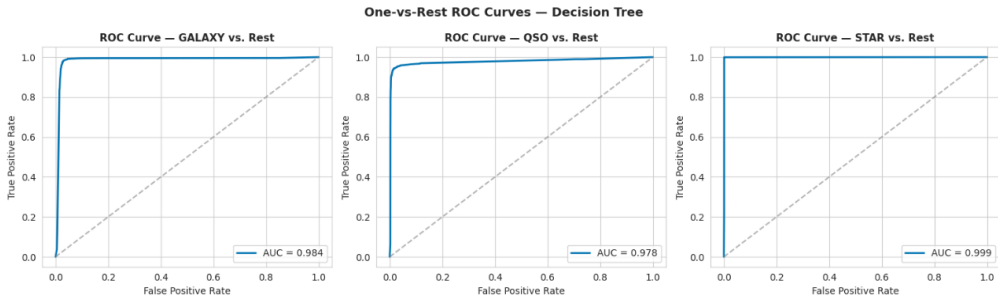
However, performance does *not* collapse toward random guessing. For three classes, the remaining photometric and colour features still allow SVM, KNN, and the Decision Tree to reach approximately **87% accuracy**.

This indicates that the models learned meaningful patterns from *u*, *g*, *r*, *i*, *z* and the engineered colour indices, rather than depending entirely on one feature.

Performance Comparison Chart



Best Performing Model – Decision Tree ROC



Interpretation & Conclusion

- **High Baseline Performance:** All evaluated models performed very well (Accuracy > 0.92). This suggests the engineered features-especially redshift and colour indices-provide extremely strong discriminative power.
- **The Winner – Decision Tree:** The Decision Tree achieved the highest Accuracy (97.71%) and Weighted F1-score (97.71%). It effectively models **non-linear relationships** and feature interactions.
- **Why the Decision Tree Excelled:** Astronomical classification relies on specific property thresholds. A decision tree naturally applies sequential, hard cutoffs (e.g., isolating objects by redshift, then refining by u-g or g-r magnitudes), perfectly mimicking the logical approach a human astronomer would use.
- **Model Drawbacks:**
 - *Logistic Regression* is constrained to linear decision boundaries.
 - *KNN* is heavily impacted by the density/scale of the geometric feature space.
 - *Naive Bayes* assumes feature independence, which severely hurts its performance since the photometric bands (u, g, r, i, z) are highly correlated.